

라즈베리파이를 이용한 안티 드론 레이저 개발

박시현 • 최동화
세종과학예술영재학교

A Study on the Development of Anti-Dron Laser Using the Raspberry Pi

Sihyeon Park • Donghwa Choi
Sejong Academy of Science and Arts

연구요약

Raspberry Pi와 Python OpenCV의 알고리즘을 이용하여 안티 드론 레이저를 제작하였다. OpenCV Tracking API를 이용하여 71% 추적 성공률을 가진 드론추적알고리즘을 구현하고 이를 본 연구에서 제작한 Raspberry Pi를 활용한 안티 드론 레이저에 적용할 수 있는 결과를 통해 Raspberry Pi로 효과적인 안티 드론 레이저 개발에 대한 가능성을 얻을 수 있었다. 라즈베리파이라는 경제적인 제어를 이용해서 효과적이고 실용적인 안티 드론 레이저, 객체 추적 장치를 만들 수 있다는 결과를 보여주었다.

중심어 : 안티 드론 레이저 트래킹 시스템, 레이저, 안티드론, 객체 인식, 객체 추적

ABSTRACT

An anti-drone laser is a laser weapon that tracks and shoots drones. It is known that many researches on the anti-drone lasers are being conducted to take advantages of high speed, high accuracy and low price. The aim of this study is to understand anti-drone technology and to realize a simple anti-drone laser tracking system by using several devices we can find easily around us. Therefore, in this study, a Raspberry Pi and OpenCV are employed to implement an anti-drone laser tracking system. To this purpose, drone tracking codes with six different algorithms are individually employed and the hardware for our anti-drone laser tracking system is configured using a commercial security camera, laser, etc. By setting three criteria for selecting algorithms: tracking speed, tracking success rate, and tracking distance and by comparing several algorithms in many experimental data, the drone tracking program including CSRT algorithm for our anti-drone laser control system is adopted. The study shows that the average tracking success rate reaches 71% when the drone tracking program with CSRT algorithm is used. It is also confirmed that our anti-drone laser tracking system is able to track an 20 cm-diameter balloon 10 m away from the system. It is highly valuable that our anti-drone laser tracking system is configured as a very simple system only by using a Raspberry Pi. This finding suggest be utilized that this control technology be applicable in many civil appliances.

Key words: anti-drone laser tracking system, anti-drone, object recognition, object tracking

I. 서론

드론의 발전과 함께 ‘안티 드론’의 중요성이 증가하고 있다. 그중에서도 비용 대비 큰 성능을 기대할 수 있는 레이저를 이용한 안티 드론 레이저는 전 세계적으로 주목받고 있다. 본 연구에서는 안티 드론 레이저를 Raspberry Pi 4를 이용하여 Python 프로그래밍으로 구현하였다. Raspberry Pi를 이용한 점에서 경제적이고 실용적인 안티 드론 레이저 개발 방법을 소개하고 안티 드론 레이저에 필요한 공학·프로그래밍적 기술을 보여준다. 안티 드론 레이저의 원리와 작동 시스템을 이해하고 카메라를 통한 객체 인식과 추적, 입력된 영상 정보를 바탕으로 한 스텝 모터의 실시간 조정, 목표 대상에 레이저 출력 등 장치 제작에 필요한 기술을 직접 구현하였으며 효과적인 성능을 내기 위한 다양한 알고리즘을 비교하며 실험하였다. 본 연구의 결과는 안티 드론 레이저뿐만 아니라 객체 추적 시스템의 개발에도 적용, 활용할 수 있다.

II. 이론적 배경

1. 안티 드론 레이저

가. 안티 드론이란 무엇인가.

최근 드론은 활발하게 발전함에 따라 군사용, 레저, 촬영, 배송 등의 다양한 용도로 활용되고 있으나 드론에 대한 대처는 드론의 확산에 비해 발전이 더딘 상황이다. 하지만 전쟁, 테러 등의 사고 위험이 커지자 안티 드론의 중요해지고 있고[1], 앞으로의 안티 드론 시장은 폭발적인 성장이 예상된다.

안티 드론 방법은 수동형과 능동형으로 나뉘며 능동형은 포획, 파괴, 전파 교란 등의 물리적인 기술이 동원되고 수동형은 법적 규제 등이 있다. 능동형 방법은 다시 ‘하드킬’, ‘소프트킬’ 방법으로 나뉘는데 하드킬은 발사체나 레이저 등으로 파괴, 손상하는 방법이고 소프트킬은 전자파를 이용한 전파 교란, 해킹 등의 방법으로 드론을 무력화한다. 연구에서는 레이저를 이용한 하드킬 기술의 안티 드론 장치를 구현하였다.

나. 레이저

한국방위산업진흥회에 따르면 최근 가장 주목받는 드론 제압 수단은 레이저라고 한다[1]. 미국을 비롯한 강대국은 안티 드론 레이저 개발에 꾸준히 힘을 쏟고 있고 한국 또한 2020년 1월 국방부의 안티 드론 레이저 대공 무기를 처음 선보이면서 기술력을 증명하였다[2]. 레이저(Laser; Light Amplification by Stimulated Emission of Radiation)는 공진기를 이용하여 전자기파를 증폭시켜 얻을 수 있는 직진성이 강한 인공적인 빛으로 직진성과 응집된 부분의 빛의 세기가 매우 강하다. 비용 대비 높은 효율이라는 강점을 보여주는 레이저는 안티 드론 레이저에서도 효과적인 성능을 보인다.

다. 안티 드론 레이저 프로세스

안티 드론 레이저의 작동 프로세스는 카메라로 원하는 표적(드론)을 찾고 추적한 뒤 레이저를 대상에 쏘는 것이다. 이를 위해서는 영상 인식 기술과 자동 모터 조절 알고리즘을 유기적으로 연결하여 계속해서 안티 드론 레이저가 작동할 수 있도록 해야 한다.

2. 객체 인식 및 추적

가. 컴퓨터 비전과 Open CV

영상 처리를 통해 컴퓨터, 기계가 시각적인 역할을 처리하도록 연구하는 분야를 컴퓨터 비전이라 한다[3]. 본 연구에서는 Python과 OpenCV 라이브러리를 통해 영상을 다루어 정보를 인식, 추출할 것이다. OpenCV(Open Source Computer Vision Library)는 실시간 영상 처리에 중점을 둔 영상 처리 라이브러리로 Python을 비롯한 다양한 언어에서

손쉽게 영상을 처리할 수 있도록 해준다.

나. 프로그래밍 관련 API

Python OpenCV로 Tracking(추적)을 하는 방법으로는 대표적으로 Tracking API를 이용하는 방법과 mean-shift, CAMshift를 이용하는 방법이 있다. mean-shift 알고리즘은 데이터 분포에서 peak, 무게중심을 찾고 그 방향으로 화면의 중심을 이동하면서(모터를 통해 조절하여) 객체를 추적해나가는 방법이다. CAMshift(Continuously Adaptive Mean Shift) 알고리즘은 mean-shift 알고리즘의 단점인 탐색 환경이 고정되어 있고 빠르게 이동하는 객체 추적이 어렵다는 점을 보완한 알고리즘으로 객체의 크기, 회전 변화를 추적할 수 있다. Tracker 클래스를 이용하는 방법은 자체 간단한 코드로 물체 선택과 추적까지 가능케 한다. TrackerKCF, TrackerMOSSE, TrackerCSRT, TrackerGOTURN 4가지의 옵션이 있는데 OpenCV의 버전에 따라 지원 여부가 다르다. KCF, MOSSE는 빠르게 작동되지만 CSRT는 느린 반면 정확하게 추적한다는 결과들이 선행 연구를 통해 알 수 있었는데 본 연구에서도 직접 확인해 볼 것이다[4, 5].

3. 라즈베리파이

가. Raspberry Pi 4

Raspberry Pi(라즈베리파이)란 초소형, 초저가의 컴퓨터로 교육용으로 개발되었고 현재 다양한 방면에서 아두이노와 함께 활발하게 사용되는 컴퓨터이다[6, 7]. Linux 체계를 따르고 보통 Raspberry Pi OS(Raspbian)를 이용한다. 작은 크기, 저렴한 가격에 비해 높은 성능을 갖고 있기 때문에 본 연구의 안티 드론 레이저를 더욱 경제적이고 실용적으로 제작할 수 있게 한다. 연구에서는 Raspberry Pi 4를 이용하였다.

III. 연구 방법 및 절차

1. 안티 드론 레이저 개발 환경 구축

가. Raspberry Pi 4 설정

Raspberry Pi 4 Model B를 이용하였다. Raspberry Pi Touch Display를 보드와 연결한다. 이때 GPIO에 연결선을 통해 power를 공급할 수 있도록 하고 보드의 DSI Display Port와 연결한다. 그리고 C타입 충전기를 USB-C에 꽂는데 1A는 약해서 Display가 깨지는 현상을 볼 수 있었다. 따라서 2A 이상의 충전기를 사용하는 것이 좋다. 나중에는 USB 포트에 편의를 위해 키보드와 마우스를 연결하고, 안티 드론 레이저의 USB 카메라를 연결한다. 다음으로 microSD를 연결하면 Raspberry Pi를 이용할 수 있다.

나. Raspberry Pi OS 설치

Raspberry Pi 4에 설치할 운영체제는 Raspberry Pi OS(이하 Raspbian)로 두 가지 방법으로 설치 가능하다. 공식 홈페이지 <https://www.raspberrypi.com/software/>에서 Raspberry Pi Imager를 설치하고 원하는 Raspbian image를 microSD카드에 넣을 수 있다. 또는 같은 웹사이트에서 operating system image를 통해 image zip파일을 설치하고 이미지를 microSD카드로 전송할 수 있는 프로그램을 이용하여 SD카드에 image를 넣을 수 있다. 본 연구에서는 두 방법 모두 시도하였고 성공하였다. 첫 번째 방법은 Raspberry Pi Imager v1.6.2를 통해 Raspberry Pi OS (32-bit)를 SD카드에 write했고, 두 번째 방법은 설치한 이미지 파일 2021-10-30-raspbian-bullseye-armhf-full.zip을 balena Etcher 프로그램을 통해 SD카드에 write했다. 그리고 SD카드를 Raspberry Pi 4에 꽂으면 정상적으로 Raspbian이 실행된다.

다. Raspberry Pi 기본 설정

Raspberry Pi가 실행되면 꼭 해야 할 것은 원격으로 컨트롤할 수 있도록 설정하는 것이다. 본 연구에서는 VNC 개념을 이용하였다. 같은 네트워크망 안에서 Raspberry Pi의 IP를 통해 연결하면 외부 컴퓨터로 Raspberry Pi 화면을

보고 마우스, 키보드 입력을 할 수 있다. 이를 위해서 외부 컴퓨터에는(연구 시 원격 조종할 컴퓨터) VNC viewer를 설치하였고, Raspberry Pi에서는 VNC 설정을 enable로 하고 비밀번호를 알고 있으면 된다. 원하는 Python 버전이 있다면 설치하거나 업그레이드를 한다. 키보드가 영미권 기준으로 default되어 있기 때문에 우리가 사용하는 일반적인 키보드를 이용하기 위해서는 설정에서 바꾸어주어야 한다.

라. Open CV 설치

Python에 cv2 module을 불러오기 위해서 OpenCV를 설치해야 한다. 그리고 cv2의 Tracker 패키지를 이용하기 위해서는 OpenCV의 확장형인 OpenCV Contrib을 설치해야 한다. 따라서 연구에서는 opencv-contrib-python을 설치하였다. OpenCV를 이용하기 위해 필요한 패키지들을 먼저 설치하고 pip의 버전과 OpenCV를 설치할 위치를 확인한 뒤 pip install opencv-contrib-python을 terminal에 입력하거나 시간을 단축하기 위해서 ~python==4.x.x.x로 특정 버전의 패키지를 설치할 수도 있다. python3에서 import cv2를 통해 설치 여부를 확인한다.

2. 안티 드론 레이저의 하드웨어 제작

안티 드론 레이저를 제작하기 위해 감시카메라를 개조하였다. 제작에 사용된 주된 부품에는 삼각대, 10W 레이저, COYAL 전동줌렌즈, 레이저 드라이브, 웹카메라, 감시카메라의 하단부분, 스텝모터, 감시카메라 상단부분, 누전차단기, 모터드라이브, 전원장치 등이 있다. 감시카메라의 tilt 부분에 레이저, 전동줌렌즈, 카메라 등을 부착하고 감시카메라 pan부분에는 새로운 모터를 부착한 후 tilt와 pan부분을 합쳐 삼각대에 설치하였다. 모터 드라이브와 전원장치를 관리하기 위해 정육면체 형태의 알루미늄 상자를 만들어 바퀴를 부착한 후 세부 드라이브와 전원장치를 부착해 control box를 만들고 이를 레이저, 카메라, 모터와 전선으로 연결하여 안티 드론 레이저를 제작하였다.

개조에 사용된 감시카메라

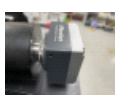
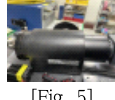


[Fig. 1] The tilt part of the surveillance camera

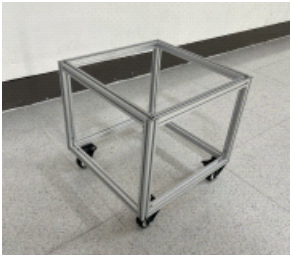
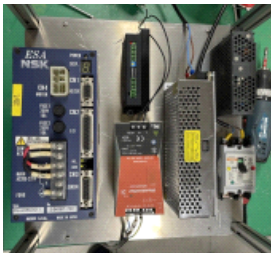


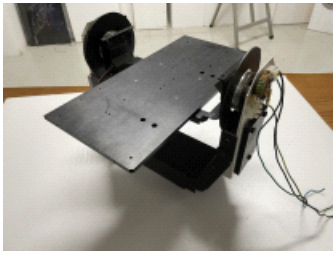
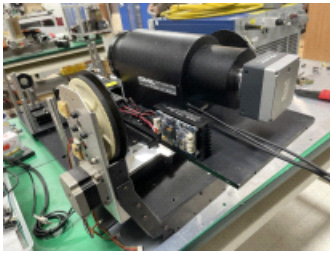
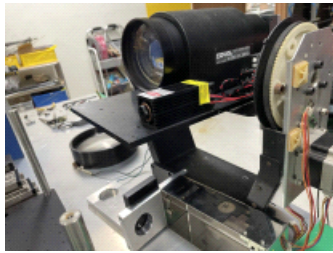
[Fig. 2] The pan part of the surveillance camera

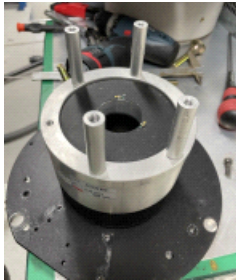
안티 드론 레이저 제작에 사용된 부품들

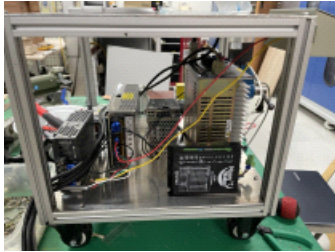
10W 레이저	카메라	전동줌렌즈	모터(회전)	모터 드라이브	전원장치	메인스위치	컨트롤 박스 프레임
							
[Fig. 3] 10W laser	[Fig. 4] Camera	[Fig. 5] Motorized zoom lens	[Fig. 6] Motor	[Fig. 7] Motor drive	[Fig. 8] Power supply	[Fig. 9] Main switch	[Fig. 10] Control box frame

제작 과정

control box 제작		
control box 프레임 제작 알루미늄판 부착, 바퀴 부착	control box에 모터 드라이브와 전원장치, 누전 차단기 설치	control box에 라즈베리파이, 모니터 설치, 전선 연결
		
[Fig. 11] Control box frame	[Fig. 12] Control box	[Fig. 13] Control box with monitor

안티드론레이저 tilt제작		
감시카메라의 tilt 부분	레이저, 전동줌인렌즈, 웹 카메라, 레이저 드라이브 설치	
		
[Fig. 14] The tilt part of the surveillance camera	[Fig. 15] Laser, electric zoom lens, web camera, laser drive installed	[Fig. 16] Laser, electric zoom lens, web camera, laser drive installed

안티드론레이저 pan 제작		
감시카메라의 pan 부분	스텝 모터 교체 <-> DD모터	pan 부분 외각 커버 부착 후 삼각대 연결
		
[Fig. 17] The pan part of the surveillance camera	[Fig. 18] DD motor	[Fig. 19] The bottom part of the anti-drone laser

안티드론레이저 제작		
pan 부분과 tilt 부분 연결	control box와 부품 전선으로 연결	안티 드론 레이저 완성
		
[Fig. 20] Connecting the pan part and the tilt part	[Fig. 21] Connecting the control box and the parts with wires	[Fig. 22] Anti-drone laser

3. 안티 드론 레이저 소프트웨어 프로그램 구현

가. 드론 추적 프로그램

본 연구에서는 OpenCV에서 제공하는 Tracking API를 이용하여 4개의 객체 추적 코드를 구현하였다. 코드에 사용된 Tracking API는 KCF(Kernelized Correlation Filters) 알고리즘을 기반으로 한 cv2.TrackerKCF_create(), 내부의 그레이 스케일을 이용한 tracker = cv2.legacy.TrackerMOSSE_create(), MIL(Multiple Instance Learning) 알고리즘을 기반으로 한 tracker = cv2.TrackerMIL_create(), CSRT(Channel and Spatial Reliability) 알고리즘을 기반으로 한 tracker = cv2.TrackerCSRT_create()이다. 객체 추적 코드는 다음과 같다.

```

1  import cv2
2  import numpy as np
3
4  cap = cv2.VideoCapture(0)
5  cap.set(cv2.CAP_PROP_FRAME_HEIGHT,640)
6  cap.set(cv2.CAP_PROP_FRAME_WIDTH,480)
7
8  if not cap.isOpened():
9      exit()
10
11 #TRACKINGAPI (선택)
12 tracker = cv2.TrackerKCF_create()
13 #tracker = cv2.legacy.TrackerMOSSE_create()
14 #tracker = cv2.TrackerMIL_create()
15 #tracker = cv2.TrackerCSRT_create()
16
17 ret, img = cap.read()
18
19 cv2.namedWindow('ROIchoose')
20 cv2.imshow('ROIchoose',img)
21
22

```

[Fig. 23] Object tracking code with tracking API

```

23 #setting ROI
24 rect = cv2.selectROI('ROIchoose', img, fromCenter=False, showCrosshair=True)
25 cv2.destroyAllWindows('ROIchoose')
26
27 # Initialize tracker
28 tracker.init(img, rect)
29
30 while True:
31     ret, rc = cap.read()
32
33     if not ret:
34         exit()
35
36     success, box = tracker.update(rc)
37
38     left, top, w, h = [int(v) for v in box]
39
40     cv2.rectangle(rc, pt1=(left, top), pt2=(left+w, top+h), color=(255,255,255),
41                  thickness=3)
42
43     cv2.imshow("img", rc)
44
45     if cv2.waitKey(1) == ord('q'):
46         break

```

[Fig. 24] Object tracking code with tracking API

본 코드에서는 line 4-6에서 웹카메라에서 실시간으로 영상을 frame으로 입력받는다. 드론을 추적하기 위해 영상처리 초기의 초점이 관찰하려는 대상에 맞춰줘야 한다. 이에 본 연구에서는 원하는 피사체를 따로 분리하여 영상처리를 지정하는 관심영역 설정, 즉 ROI(Region of Interest)를 이용하였다. 코드의 line20~25에서는 웹카메라에서 입력받는 frame을 윈도우창에 띄우고 cv2.selectROI라는 함수로 마우스로 원하는 영역을 설정할 수 있게 한다 그 후 코드에서 설정한 ROI를 매 프레임을 새롭게 입력받을 때 마다 tracker.update()함수를 이용해 구역의 좌표값을 찾고 그 RIO영역을 cv2.rectangle()함수를 이용해 네모칸을 frame위에 그려 윈도우창에 출력을 한다. 그러면 실시간으로 선택한 RIO 영역의 피사체에 따라 네모칸이 그려지며 추적이 진행된다.



[Fig. 25] Setting ROI (region of interest)



[Fig. 26] Tracking the ROI (region of interest) selected area

OpenCV에서 제공하는 Tracking API 이외에도 영상 객체 추적에 주되게 사용되는 Camshift 알고리즘을 이용한 객체 추적 코드를 구현하였다. 그리고 Camshift 알고리즘을 보완한 Meanshift 알고리즘을 이용한 객체 추적 코드 또한 구현하였다. 코드는 다음과 같다.

```
1 import numpy as np
2 import cv2
3
4 cap = cv2.VideoCapture(0)
5 # Take first frame of the video
6 ret, frame = cap.read()
7
8 if not cap.isOpened():
9     exit()
10
11 cv2.namedWindow('ROIchoose')
12 cv2.imshow('ROIchoose', frame)
13
14 # Setting ROI
15 rect = cv2.selectROI('ROIchoose', frame, fromCenter=False, showCrosshair=True)
16 cv2.destroyAllWindows('ROIchoose')
17 x, y, w, h = rect
18
19 cap = cv2.VideoCapture(0)
20 # If not cap.isOpened():
21 #     exit()
22
23 # 초기 사각형 범위 : (x, y, w, h)
24 rc = (x, y, w, h)
```

[Fig. 27] Object tracking code using the Meanshift algorithm

```
26 ret, frame = cap.read()
27
28 if not ret:
29     print('frame read failed!')
30     exit()
31
32 roi = frame[y:y+h, x:x+w]
33 roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
34
35 # HS 히스토그램 계산
36 channels = [0, 1]
37 ranges = [0, 180, 0, 256]
38 hist = cv2.calcHist([roi_hsv], channels, None, [90, 128], ranges)
39
40 # Camshift 알고리즘 종료 기준
41 term_crit = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)
42
43 # 이미지 상 프레임 처리
44 while True:
45     ret, frame = cap.read()
46
47     if not ret:
48         break
```

[Fig. 28] Object tracking code using the Meanshift algorithm

```
50
51 # HS 히스토그램에 대한 역투영
52 frame_hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
53 backproj = cv2.calcBackProject([frame_hsv], channels, hist, ranges, 1)
54
55 # Camshift
56 ret, rc = cv2.CamShift(backproj, rc, term_crit)
57
58 # 추적 결과 화면 출력
59 cv2.rectangle(frame, rc, (0, 0, 255), 2)
60 # 타겟 그려는 함수, 사각형 목표 루어지면 사각형에 내접하는 타원형 그려줌
61 cv2.ellipse(frame, rc, (0, 255, 0), 2)
62 cv2.imshow('frame', frame)
63
64 if cv2.waitKey(60) == 27:
65     break
66
67 cap.release()
68 cv2.destroyAllWindows()
```

[Fig. 29] Object tracking code using the Meanshift algorithm

```
1 import numpy as np
2 import cv2
3
4 cap = cv2.VideoCapture(0)
5 # Take first frame of the video
6 ret, frame = cap.read()
7
8 cv2.namedWindow('ROIchoose')
9 cv2.imshow('ROIchoose', frame)
10
11 # Setting ROI
12 rect = cv2.selectROI('ROIchoose', frame, fromCenter=False, showCrosshair=True)
13 cv2.destroyAllWindows('ROIchoose')
14 x, y, w, h = rect
15
16 # Setup initial location of window
17 track_window = (x, y, w, h) # simply hardcoded the values
18
19 # set up the ROI for tracking
20 roi = frame[y:y+h, x:x+w]
21 haw_roi = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
22 haw = cv2.cvtColor(haw_roi, np.array((0, 0, 180, 256)), np.array((0, 0, 180, 256)))
23 roi_hist = cv2.calcHist([haw_roi], [0], None, [180, 256], [0, 180])
24 cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)
25
26 # Setup the termination criteria, either 10 iteration or more by atleast 1 pt
27 term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1 )
```

[Fig. 30] Object tracking code using the Camshift algorithm

```
26 while(1):
27     ret, frame = cap.read()
28
29     if ret on True:
30         haw = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
31         dst = cv2.calcBackProject([haw], [0], roi_hist, [0, 180], 1)
32         # apply meanshift to get the new location
33         ret, track_window = cv2.meanShift(dst, track_window, term_crit)
34         # Draw it on image
35         x,y,w,h = track_window
36         img2 = cv2.rectangle(frame, (x,y), (x+w,y+h), 255,2)
37         cv2.imshow('img2',img2)
38         k = cv2.waitKey(60) & 0xFF
39         if k == 27:
40             break
41         else:
42             cv2.imwrite(chr(k)+".jpg",img2)
43     else:
44         break
45 cv2.destroyAllWindows()
46 cap.release()
```

[Fig. 31] Object tracking code using the Camshift algorithm

모터 구동 프로그램과 레이저 출력 프로그램

```

1 import time
2 import RPi.GPIO as GPIO
3
4 GPIO.setmode(GPIO.BCM)
5 GPIO.setup(18, GPIO.IN)
6 GPIO.setup(18, GPIO.OUT)
7
8 try:
9     while True:
10         GPIO.output(18, GPIO.HIGH)
11         time.sleep(0.1)
12         GPIO.output(18, GPIO.LOW)
13         time.sleep(0.1)
14
15 except KeyboardInterrupt:
16     GPIO.output(GPIO.LOW)
17 finally:
18     GPIO.cleanup()
19

```

[Fig. 32] Motor driving code

```

1 import time
2 import RPi.GPIO as GPIO
3
4 GPIO.setmode(GPIO.BCM)
5 GPIO.setup(27, GPIO.IN)
6 GPIO.setup(27, GPIO.OUT)
7
8 if cv2.waitKey(1) == ord('a'):
9
10 try:
11     if cv2.waitKey(1) == ord('a'): # a를 누르면 레이저 출력
12         while True:
13             GPIO.output(27, GPIO.HIGH)
14             if cv2.waitKey(1) == ord('a'): # a를 누르면 레이저 중지
15                 GPIO.output(27, GPIO.LOW)
16
17 finally:
18     GPIO.cleanup()
19

```

[Fig. 33] laser output code

4. 안티 드론 레이저 개발

가. 드론 추적 알고리즘 선정

본 연구에서 최적화된 객체 추적 코드를 선정하기 위해 드론을 추적할 때 발생할 수 있는 5가지 상황과 유사한 영상 5개를 촬영하여 각각 객체 추적 코드에 입력해 객체 추적 성공률을 구해 보았다. 드론을 추적할 때 드론의 속력이 빨라지는 경우, 레이저와 가까워지거나 멀리 이동하는 경우, 건물 또는 나무 등의 방해물에 가려지는 경우 등의 다양한 상황이 있다. 이에 이와 비슷한 상황을 풍선에 적용하여 영상을 촬영하였다.

1) 실험과정

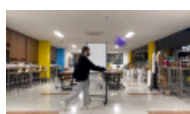
가) 카메라, 풍선, 막대, 스크린을 준비한다.

나) 보라색 풍선을 이용해 각 상황별 영상을 촬영한다.

- 상황1: 풍선이 움직이는 영상
- 상황2: 풍선이 갑자기 빠르게 움직이는 영상
- 상황3: 풍선이 멀어졌다가 가까워지는 영상
- 상황4: 풍선이 화면 밖에 나갔다가 다시 들어오는 영상
- 상황5: 풍선이 스크린에 가려지는 영상



[Fig. 34] Situation1



[Fig. 35] Situation2



[Fig. 36] Situation3



[Fig. 37] Situation4



[Fig. 38] Situation5

다) 6개의 객체 추적 코드를 다음과 같이 수정하여 총 프레임수와 객체 추적이된 프레임수를 출력한다.

```

14 # initialize tracker
15 tracker.init(img, rect)
16
17 # 총 프레임수와 객체 추적된 프레임수 출력
18 frame=0
19 count=0
20
21 while True:
22     ret, rc = cap.read()
23     frame=frame+1

```

[Fig. 39] Experimental code

```

31 while True:
32     ret, rc = cap.read()
33     frame=frame+1
34
35     if not ret:

```

[Fig. 40] Experimental code

```

37 success, box = tracker.update(rc)
38
39 if success:
40     count=count+1
41
42 left, top, w, h = [int(v) for v in box]
43

```

[Fig. 41] Experimental code

이때 [Fig. 39]와 같이 총 프레임수를 저장할 frame 변수, 객체 추적되는 프레임수를 저장할 count 변수를 지정한다.

그 후 프레임을 불러올 때마다 [Fig. 40]와 같이 frame 변수에 1을 더하고 트래킹이 성공할 때마다 [Fig. 41]와 같이 count 변수에 1을 더하도록 코드를 수정하였다.

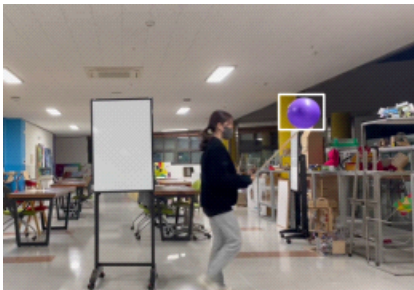
2) 실험 결과

추적 성공률을 $\frac{\text{추적된 프레임수}(\text{count})}{\text{총 프레임수}(\text{frame})} \times 100(\%)$ 라 하면 아래 표와 같은 결과를 구할 수 있었다.

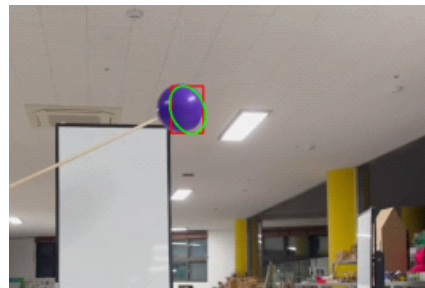
<Table 1> The result of Object tracking success rate by algorithm

알고리즘	상황 1	상황2	상황 3	상황 4	상황 5
tracker = cv2.legacy.TrackerMOSSE_create()	28	36	0	0	3
tracker = cv2.TrackerMIL_create()	100	47	100	100	100
tracker = cv2.TrackerCSRT_create()	100	78	100	20	57
tracker = cv2.TrackerKCF_create()	24	37	5	5	54
meanshift	57	38	48	61	55
camshift	100	45	100	15	100

추적 성공률 뿐만 아니라 각 알고리즘 종류에 따라 추적 속도, 다른 곳을 추적하는 오류 발생 횟수, 추적 가능 속도, 추적 가능 거리 등에 차이가 있었다. tracker = cv2.legacy.TrackerMOSSE_create()를 이용한 객체 추적 코드 같은 경우는 추적 속도는 매우 빠르나 추적 성공률이 전체적으로 매우 낮으며 풍선의 이동 속력이 너무 빠르거나 거리가 멀면 측정을 하지 못했다. tracker = cv2.TrackerMIL_create()를 이용한 객체 추적 코드같은 경우는 첫 번째 코드에 비해 추적 속도가 느렸지만 빠른 편에 속했다. 그리고 적당한 속력으로 적당한 거리에 있는 물체는 잘 측정하며 멀리있는 물체 또한 잘 추적하였으나 상황 2와 같이 빠른 속력으로 이동할 경우 추적을 잘 하지 못하였으며 상황 4와 상황 5와 같이 가려지거나 화면에 없다가 다시 생기는 경우에 추적을 잘하지 못하였다. 또한, 실험 4에서는 추적이 풍선이 아닌 천장 등 다른 곳에 인식되기도 했다. tracker = cv2.TrackerCSRT_create()를 이용한 객체 추적 코드는 추적 속도가 보통이었으며 전체적으로 추적 성공률이 높았다. 상황 5의 추적 성공률에서 볼 수 있듯이 방해물이 존재해도 객체 추적이 잘 이루어졌으나 풍선 전체가 가려질 경우에는 추적을 잘 하지 못했다. 또, 멀리 있는 풍선을 잘 측정할 수 있었고 상황 4에서 풍선이 프레임 밖으로 나갔다가 다시 등장하는 경우에 풍선을 다시 추적하였다. tracker = cv2.TrackerKCF_create()를 이용한 객체 추적 코드는 전반적으로 추적 성공률이 매우 낮았다. 추적 속도가 빠르나 물체를 잘 추적하지 못했다. 특히, 풍선의 속력이 빠르거나 방해물에 의해 가려지면 추적을 하지 못했다. Meanshift 알고리즘을 이용한 객체 추적 코드는 추적 속도가 느렸으며 풍선이 아닌 다른 물체를 추적하는 경우가 다수였다. 그래서 추적 성공률이 높아보이나 대부분이 천장, 의자 등 주변의 다른 물체를 추적한 경우에 의한 성공률이므로 추적 성공률이 낮다고 판단할 수 있다. Camshift를 이용한 객체 추적 코드는 전반적인 추적 성공률이 높았다. 다만 추적 속도가 매우 느렸으며 풍선의 속력이 빠르면 추적이 잘 안된다는 점과 주변의 환경을 추적하고, 장애물에 의해 일부가 가려지면 추적이 안되는 등 주변환경에 영향을 많이 받는 것을 확인할 수 있었다.



[Fig. 42] Tracking balloons using the CSR algorithm



[Fig. 43] Tracking balloons with the Camshift algorithm

본 연구에서 안티 드론 레이저는 다양한 환경에서 드론을 실시간으로 추적하고 레이저를 발사해야하기 때문에

정확도가 높고 추적속도가 빠른 알고리즘이 필요하다. 따라서 빠른 추적 속도, 높은 추적 성공률, 원거리 추적 기능 다음 세가지를 알고리즘 선정 기준으로 정하였다. 그 결과 본 연구의 드론 추적 코드로 비교적 추적 속도가 빠르고, 빠르게 이동하는 물체나 장애물에 의해 일부가 가려지는 물체를 잘 측정할 수 있는 `tracker = cv2.TrackerCSRT_create()`을 이용한 객체 추적 코드를 선정하였다.

나. 라즈베리파이와 안티드론레이저 연결 후 작동

라즈베리파이에 드론 추적 코드, 모터 구동 코드, 레이저 출력 코드, 카메라 출력 코드를 저장한 후 안티드론레이저와 연결한다. 그 후 안티드론레이저를 구동시킨다. 본 연구에서 개발한 안티드론의 모습은 다음과 같다.



[Fig. 44] Anti-drone laser



[Fig. 45] Anti-drone laser running



[Fig. 46] Anti-drone laser laser output

다. 안티드론레이저 구동 및 보완

본 연구의 안티드론레이저를 구동해보며 성능을 분석한 결과 드론이 화면 밖을 나가거나 멀리 있으면 추적이 안되는 문제점을 인식하였다. 이를 보완하기 위해 줌인렌즈를 도입하여 드론이 화면 밖을 나가면 줌 아웃을 하여 다시 드론을 추적할 수 있도록 하였다. 다음은 줌인렌즈를 제어하는 코드이다. 이때 줌 인 또는 줌 아웃을 할 때 초점이 변하기 때문에 초점 또한 제어할 수 있게 하였다. 최종적으로 줌인렌즈를 도입한 안티드론레이저는 10m 부근 멀리까지 드론을 추적할 수 있었으며, 20cm의 지름의 물체도 레이저로 추적할 수 있는 것을 확인하였다.

```

1 import time
2 import RPi.GPIO as GPIO
3 import cv2
4
5 capture=cv2.VideoCapture(0)
6
7 GPIO.setmode(GPIO.BCM)
8 GPIO.setup(18, GPIO.IN)
9 GPIO.setup(18, GPIO.OUT)
10 GPIO.setup(17, GPIO.IN)
11 GPIO.setup(17, GPIO.OUT)
12 GPIO.setup(23, GPIO.IN)
13 GPIO.setup(23, GPIO.OUT)
14 GPIO.setup(22, GPIO.IN)
15 GPIO.setup(22, GPIO.OUT)
16
17 while cv2.waitKey(33) != ord('q'):
18     ret, frame=capture.read()
19     cv2.imshow("Videoframe", frame)
20     if cv2.waitKey(100) == ord('a'):
21         GPIO.output(18, GPIO.HIGH)
22

```

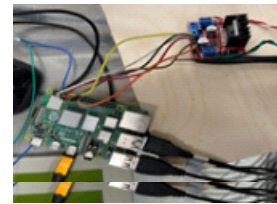
[Fig. 47] Zoominlens control code

```

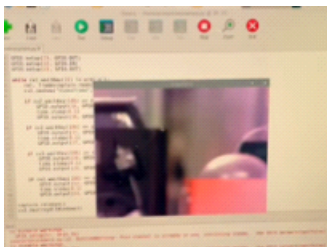
23     time.sleep(0.1)
24     GPIO.output(18, GPIO.LOW)
25
26     if cv2.waitKey(100) == ord('s'):
27         GPIO.output(17, GPIO.HIGH)
28         time.sleep(0.1)
29         GPIO.output(17, GPIO.LOW)
30
31     if cv2.waitKey(100) == ord('z'):
32         GPIO.output(23, GPIO.HIGH)
33         time.sleep(0.1)
34         GPIO.output(23, GPIO.LOW)
35
36     if cv2.waitKey(100) == ord('x'):
37         GPIO.output(22, GPIO.HIGH)
38         time.sleep(0.1)
39

```

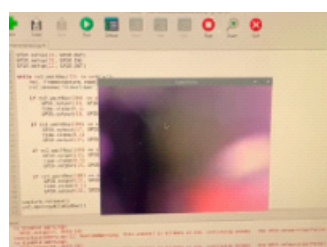
[Fig. 48] Zoominlens control code



[Fig. 49] Zoominlens and raspberry pie connected



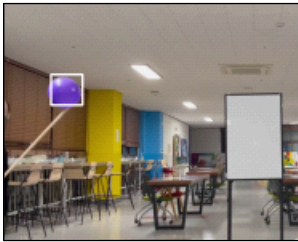
[Fig. 50] The screen is zooming in



[Fig. 51] The screen is zooming in

IV. 연구 결과

본 연구에서는 라즈베리파이 4, OpenCV를 이용하여 안티 드론 레이저를 제작하였다. 안티 드론 레이저는 OpenCV에서 제공하는 Tracking API인 CSRT 알고리즘을 기반으로한 cv2.TrackerCSRT_create()을 함수를 이용해 드론을 추적하는 코드를 구현하였다. 드론을 추적할 때의 발생할 수 있는 5가지 상황을 영상으로 촬영해 추적 성공률을 구한 결과 각 상황별로 100%, 78%, 100%, 20%, 57%이었고 평균적으로 71%의 성공률을 가지는 것을 확인하였다. 객체 추적 속도가 빨라 실시간으로 영상 처리가 가능한 것 확인하였고, 드론 추적 코드 뿐만 아니라 모터 구동 코드, 레이저 출력 코드, 카메라 구동 코드를 구현하였고 감시카메라를 개조하여 안티 드론 레이저의 하드웨어를 제작하였다. 최종적으로 카메라에 활용되는 드론을 디스플레이에서 객체 추적 설정을 하면 객체 추적 코드에 의해 추적이 되며 이 추적되는 드론의 이동 범위에 따라 모터가 이동하고 키를 누르면 레이저를 출력하여 드론을 저격할 수 있는 안티드론레이저를 개발하였다. 이때 줌인렌즈를 도입하여 안티드론레이저는 최종적으로 10m 부근에 위치한 드론까지 잘 추적하였고, 이때 20cm 반경의 물체를 레이저로 추격할 수 있었다.



[Fig. 52] Object tracking



[Fig. 53] Anti-drone laser



[Fig. 54] Anti-drone laser

V. 고찰

본 연구에서는 Raspberry Pi와 Python OpenCV 라이브러리를 이용하여 안티 드론 레이저를 제작하였다. Raspberry Pi 4는 객체 인식 및 추적 알고리즘을 원활하게 구동하였고 이를 통해 저렴한 소형 컴퓨터 Raspberry Pi를 이용한 효과적인 안티 드론 레이저 개발에 대한 가능성을 얻을 수 있었다.

추적성공률을 정의하고 드론 추적을 위한 알고리즘 선정 기준에 따라 OpenCV Tracking API의 CSRT 알고리즘을 이용한 드론 추적 코드를 안티 드론 레이저에 적합한 알고리즘으로 선정하였다. 안티 드론 레이저에 줌인렌즈를 도입함으로써 정확도를 향상시켰다. 이를 통해 안티 드론 레이저의 추적률은 71% 성공률이라는 값으로 결과를 입증하였고 10m부근의 원거리 물체도 추적 가능한 뛰어난 객체 인식 성능을 바탕으로 드론 격추 기능을 수행하게 된다.

본 연구를 통해 안티 드론 레이저에 적용되는 기술과 원리를 이해할 수 있다. 객체 인식 및 추적 기술을 이해하고 알고리즘을 구현하였으며 레이저를 이용하여 드론 무력화 기술을 적용하였다. 안티 드론 레이저의 기본 골격은 객체 추적기이다. Raspberry Pi를 이용했을 때 비용 대비 높은 추적률을 갖는 객체 추적기를 제작하였다.

VI. 결론

안티 드론 레이저는 앞으로 발전 가능성이 높은 방위 산업의 장치로서 그 원리를 이해하고 구현하는 것은 큰 의미가 있다. 본 연구에서는 Raspberry Pi 4와 스텝 모터, 레이저를 이용해 제작하였고 Python OpenCV를 기반으로 객체 추적 및 레이저 발사 알고리즘을 구현하였다. 71%의 추적 성공률을 보였으며 이는 Raspberry Pi라는 경제적인 재료를 이용해서 효과적이고 실용적인 안티 드론 레이저, 객체 추적 장치를 만들 수 있다는 결과를 보여준다. 그리고 안티 드론 레이저를 직접 개발하는 과정에서 객체 인식 및 추적 알고리즘과 alt-azm 구조의 하드웨어를 제작하는

공학적 원리를 알 수 있다.

본 연구에는 Raspberry Pi의 느린 연산 처리 속도, 카메라와 모터 성능의 부정확도, 객체 추적 및 인식 코드의 부정확함 등의 한계점이 존재하였으나 인공지능(AD)를 이용해 드론을 학습하고 인식할 수 있도록 하여 더 높은 정확도를 가진 안티 드론 레이저 개발할 수 있다. 또 안티 드론 레이저 외에도 Raspberry Pi를 이용한 저비용 ‘객체 추적기 제작 키트’를 개발 가능하다.

■ 참고문헌

- [1] 최현호. (2016). 시설보안을 넘어 안보에도 중요해진 안티 드론 기술. 국방과 기술, (451), 38-45
- [2] 스마트 국방 ‘안티드론 레이저 무기’ 첫 선. 아시아경제. 2020.01.21
- [3] 윤대회. (2019). 『C#과 파이썬을 활용한 OpenCV4 프로그래밍』 위키북스
- [4] 장일식, 강인구, 박구만. (2012). 다양한 객체 추적 알고리즘의 비교 시스템 구현. 한국통신학회 학술대논문집, 0, 734-735.
- [5] 송기범, 양용준, 이상구. (2017). 라즈베리 파이를 이용한 OpenCV 기반 CMT 알고리즘의 객체 추적 기법. 한국정보처리학회 2017년도 추계학술발표대회 2017 No.v 01, pp.895 - 898
- [6] 송주환. (2020). 라즈베리 파이를 이용한 차량 추적 시스템에 관한 연구. 한국디지털콘텐츠학회 논문지, 21(5), 1009-1016.
- [7] 김민철. (2019). 라즈베리파이를 이용한 영상신호처리기 구현. 대한전자공학회 학술대회, 448-449.
- [8] 양정연. (2020). 사용자 추적 기능을 가진 야외용 테니스 훈련용 장치 개발. 한국콘텐츠학회논문지, 20(3), 424-431.
- [9] 강현중, 이광형. (2005). 이동 객체 감시를 위한 실시간 객체추출 및 추적 시스템. 한국컴퓨터정보학회논문지, 10(2), 59-68.
- [10] 장정화. (2010). 실시간 영상에서 객체 추출 및 추적에 관한 연구. 한국산학기술학회 학술대논문집, 0, 475-478.
- [11] 백인호, 임재현, 박경주, 백준기. (2008). Pan-Tilt-Zoom 카메라를 이용한 파노라마 배경 생성과 객체 추적. 전자공학회논문지-SP, 45(3), 55-63.

■ 연구진

연번	역할	이름 (영문명)	소속	이메일
1	주저자, 교신저자	박시현(Sihyeon Park)	세종과학예술영재학교	sihyeon1024@sasa.hs.kr
2	공동연구자	최동화(Donghwa Choi)	세종과학예술영재학교	astatine@sasa.hs.kr
3	지도교사	박재형(Jaehyung Park)	세종과학예술영재학교	samzigi@sasa.hs.kr